

ARMP Release Binder

- [1. Executive Summary & Release Status](#)
- [AR Match Pro — Release ARMP-2026.07.21-PILOT-AUTH-R3](#)
- [2. Manifest](#)
- [ARMP Final Project Manifest — ARMP-2026.07.21-PILOT-AUTH-R3](#)
- [3. Product Truth](#)
- [ARMP Product Truth — R3](#)
- [4. Architecture](#)
- [ARMP Architecture — R3](#)
- [5. Security Remediation \(Authentication Design\)](#)
- [ARMP Security Remediation — R2 → R3](#)
- [6. Pilot Access Workflow / Operations Console Guide](#)
- [ARMP Pilot Access — Operations Console Guide](#)
- [7. Supabase Owner Setup](#)
- [ARMP Supabase Owner Setup — Staging](#)
- [8. Support Procedures](#)
- [ARMP Support Procedures — R3](#)
- [9. Authentication Test Report](#)
- [ARMP Authentication Test Report — R3](#)
- [10. RLS Test Report](#)
- [ARMP RLS Test Report — R3](#)
- [11. Data Boundary Test Report](#)
- [ARMP Data-Boundary Test Report — R3](#)
- [12. Test Results](#)
- [ARMP Test Results — ARMP-2026.07.21-PILOT-AUTH-R3](#)
- [13. Prohibited-Pattern Sweep](#)
- [ARMP Prohibited-Pattern Sweep — R3](#)
- [14. Deployment](#)
- [ARMP Deployment — R3](#)
- [15. Rollback](#)
- [ARMP Rollback — R3](#)
- [16. Incident Response](#)
- [ARMP Incident Response — R3 \(staging\)](#)
- [17. Claims Register](#)
- [ARMP Claims Register — R3](#)
- [18. Consistency Matrix](#)
- [ARMP Consistency Matrix — R3](#)
- [19. File Audit](#)
- [ARMP File Audit — R3 \(authoritative file per purpose\)](#)
- [20. Known Limitations & Remaining Blockers](#)
- [ARMP Known Limitations — R3](#)
- [21. Staging Acceptance Report](#)
- [ARMP Staging Acceptance Report — R3](#)

AR Match Pro — Release Documentation Binder

ARMP-2026.07.21-PILOT-AUTH-R3 · Server-Controlled Pilot Access · Confidential

Release status

Controlled real-data pilot: NOT APPROVED until staging acceptance passes and independent review completes. **Paid production: BLOCKED.** Public website / synthetic Demo Mode may proceed after staging smoke testing. Staging deployment and the 16-step E2E are **NOT YET RUN**.

Contents

- 1. Executive Summary & Release Status
- 2. Manifest
- 3. Product Truth
- 4. Architecture
- 5. Security Remediation (Authentication Design)
- 6. Pilot Access Workflow / Operations Console Guide
- 7. Supabase Owner Setup
- 8. Support Procedures
- 9. Authentication Test Report
- 10. RLS Test Report
- 11. Data Boundary Test Report

12. Test Results
13. Prohibited-Pattern Sweep
14. Deployment
15. Rollback
16. Incident Response
17. Claims Register
18. Consistency Matrix
19. File Audit
20. Known Limitations & Remaining Blockers
21. Staging Acceptance Report

1. Executive Summary & Release Status

AR Match Pro — Release ARMP-2026.07.21-PILOT-AUTH-R3

AR Match Pro — Server-Controlled Pilot Access

Release ARMP-2026.07.21-PILOT-AUTH-R3 · Revised 2026-07-22 · Phase 3 (Supabase Auth + RLS + Edge Functions + TOTP MFA)

What this release is

R3 replaces the R2 client-side, format-only pilot code with **server-controlled pilot access**: Supabase Auth (email/password + mandatory TOTP MFA), PostgreSQL with Row-Level Security, and privileged Edge Functions administered through the ARMP Operations Console. Customer financial data continues to be processed **locally in the browser** and is never sent to Supabase.

Release status STAGING VALIDATION PENDING

- **Public website / Demo Mode:** may go to production after staging smoke testing. Demo Mode is synthetic-only.
- **Controlled real-data pilot: NOT approved** until the staging acceptance battery (migrations deployed, Edge Functions deployed, RLS, MFA, invitation flow, pilot date/suspension/revocation enforcement, organization isolation, data-boundary, and the 16-step E2E) passes against the supplied Supabase staging project **and** an independent review is complete.
- **Paid production: BLOCKED** pending independent security review, attorney-approved legal documents, cleared-funds activation, production Supabase project, production SMTP, backup/recovery validation, E&O/cyber insurance, approved incident response, and production monitoring.

Verified locally (this build)

- 12 migrations apply cleanly to real PostgreSQL 16; **RLS suite 26/26** on real Postgres.
- 20 Edge Functions type-check under Deno with real dependency resolution.
- Full JS battery **14 suites, 1,485 assertions** — including the **503 R2 assertions unweakened**, frontend-security, network-instrumented data-boundary, and R2-localStorage-cannot-grant-access suites.
- Expanded prohibited-pattern gate (context-aware; secret **values** vs variable **names**; `service_role` permitted only in backend).

NOT YET RUN / NOT VALIDATED IN STAGING

No Edge Function has executed against the live project; MFA, invitations, and session revocation are unproven until the 16-step staging E2E runs. Nothing has been deployed; the staging database is untouched; `main` is untouched; no secret appears in any file.

Deploy & test

Frontend: upload the contents of `deploy/` to Netlify staging (see ARMP_DEPLOYMENT). Backend: apply via the manual GitHub Actions workflows after owner approval (see ARMP_SUPABASE_OWNER_SETUP). Tests: `npm install && npm test`.

2. Manifest

ARMP Final Project Manifest — ARMP-2026.07.21-PILOT-AUTH-R3

Package planes

- `deploy/` — Netlify-uploadable static frontend (site, Demo, pilot app, Operations Console, `/auth/`).
- `supabase/` — config, 12 migrations, 20 Edge Functions, RLS + local test shim.
- `tests/` — 14 suites (503 R2 + R3 auth/security/data-boundary/gate) + fixtures (customer-derived; never deploy).
- `docs/` — PDF documentation (below) as the primary deliverable.
- `source/internal/` — machine-readable Markdown sources + generators (not the primary docs).
- `.github/workflows/` — manual staging deploy + test workflows.

PDF documents (source → PDF, verified)

Source (source/internal)	PDF (docs/)	Generator	Pages	Text extraction	SHA-256 (short)
README.md	docs/release/README.pdf	build_pdfs.py	1	PASS	91332abc4a52c290...
ARMP_CHANGELOG.md	docs/release/ARMP_CHANGELOG.pdf	build_pdfs.py	1	PASS	5b1e8cfb69bfa0ff...
ARMP_KNOWN_LIMITATIONS.md	docs/release/ARMP_KNOWN_LIMITATIONS.pdf	build_pdfs.py	1	PASS	7bc3f56b77115048...
ARMP_ROLLBACK.md	docs/release/ARMP_ROLLBACK.pdf	build_pdfs.py	1	PASS	bd0aaac5f75e7a26...
ARMP_DEPLOYMENT.md	docs/release/ARMP_DEPLOYMENT.pdf	build_pdfs.py	1	PASS	d14d175304177a8a...
ARMP_PRODUCT_TRUTH.md	docs/release/ARMP_PRODUCT_TRUTH.pdf	build_pdfs.py	1	PASS	c30444bcf340801f...
ARMP_CLAIMS_REGISTER.md	docs/release/ARMP_CLAIMS_REGISTER.pdf	build_pdfs.py	1	PASS	9653a8e99d2f931a...
ARMP_FILE_AUDIT.md	docs/release/ARMP_FILE_AUDIT.pdf	build_pdfs.py	1	PASS	5c0a11ba7d2d82e7...
ARMP_CONSISTENCY_MATRIX.md	docs/release/ARMP_CONSISTENCY_MATRIX.pdf	build_pdfs.py	1	PASS	a1b2712432d4572f...
ARMP_ARCHITECTURE.md	docs/architecture/ARMP_ARCHITECTURE.pdf	build_pdfs.py	1	PASS	773d63f8a90ad3aa...
ARMP_SECURITY_REMEDIATION.md	docs/security/ARMP_SECURITY_REMEDIATION.pdf	build_pdfs.py	1	PASS	bbcd9657a26b73e4...
ARMP_PILOT_ACCESS_ADMIN_GUIDE.md	docs/operations/ARMP_PILOT_ACCESS_ADMIN_GUIDE.pdf	build_pdfs.py	1	PASS	fd7f786e4fea564f...
ARMP_SUPABASE_OWNER_SETUP.md	docs/operations/ARMP_SUPABASE_OWNER_SETUP.pdf	build_pdfs.py	1	PASS	a31a0fbe812fe529...
ARMP_SUPPORT_PROCEDURES.md	docs/operations/ARMP_SUPPORT_PROCEDURES.pdf	build_pdfs.py	1	PASS	9a718d7d75c0f980...
ARMP_INCIDENT_RESPONSE.md	docs/security/ARMP_INCIDENT_RESPONSE.pdf	build_pdfs.py	1	PASS	32f02e314815d5b2...
ARMP_AUTHENTICATION_TEST_REPORT.md	docs/security/ARMP_AUTHENTICATION_TEST_REPORT.pdf	build_pdfs.py	1	PASS	90a44ef4efe33caa...
ARMP_RLS_TEST_REPORT.md	docs/security/ARMP_RLS_TEST_REPORT.pdf	build_pdfs.py	1	PASS	6dbc15158b8ab95e...
ARMP_DATA_BOUNDARY_TEST_REPORT.md	docs/security/ARMP_DATA_BOUNDARY_TEST_REPORT.pdf	build_pdfs.py	1	PASS	03c59ef0df441ee3...
ARMP_STAGING_ACCEPTANCE_REPORT.md	docs/release/ARMP_STAGING_ACCEPTANCE_REPORT.pdf	build_pdfs.py	1	PASS	ff9ac722e4dc8215...

Authoritative file per purpose

See ARMP_FILE_AUDIT. One active app, console, user guide, security sheet, order-form draft, pilot-agreement draft, import template, product truth, claims register, known limitations, consistency matrix, architecture spec.

Status

Real-data pilot NOT approved until staging acceptance passes; paid production BLOCKED. Full SHA256SUMS.txt accompanies the ZIP.

3. Product Truth

ARMP Product Truth — R3

- **Access model:** invitation-only, server-controlled pilots. No self-service signup for real data; no format-only codes; no client-side entitlement. Demo Mode is synthetic-only, no expiry, no entitlement.
- **Authentication:** Supabase email/password + mandatory TOTP MFA (AAL2). Operations Console requires an active internal role — an @armatchpro.com email alone grants nothing.
- **Data boundary:** invoice/bank/remittance/delivery/matching/allocation/export/OCR content stays in the browser. Supabase stores control-plane metadata only.
- **Commercial:** Pro \$4,599/mo billed annually (\$55,188/yr) + Custom; ACH/wire only; no Stripe, no cards, no self-serve checkout. 30-Day Shadow Pilot precedes purchase. Pricing + legal are DRAFT pending attorney review.
- **Revocation:** server-side; up to 300 s to propagate to an open session.
- **Status:** real-data pilot NOT approved until staging acceptance + independent review; paid production BLOCKED.

4. Architecture

ARMP Architecture — R3

System shape

Two planes with a strict boundary: - **Data plane (browser-local)**: invoice/bank/remittance/delivery import, matching, allocation, exception review, and posting-export all run in the customer's browser. This data is never transmitted to ARMP or Supabase during the core workflow. - **Control plane (Supabase)**: authentication identity, profiles, organization membership, customer roles, internal ARMP roles, pilot entitlements, invitation metadata, MFA enrollment metadata, account status, nonfinancial security/administrative events, and login timestamps.

Components

- **Netlify** — static hosting for `deploy/` (public site, Demo Mode, the pilot app shell, the Operations Console, and `/auth/` pages).
- **Supabase Auth** — email/password + mandatory TOTP MFA (AAL2). Invitation-only; public signup disabled.
- **Supabase PostgreSQL + RLS** — control-plane tables; default-deny RLS on every protected table.
- **Supabase Edge Functions** — all privileged mutations run here with the service-role key, which exists only in the function environment.

Authorization decision

`authorize_pilot_session()` (SECURITY DEFINER) is the single source of truth for real-data access. It evaluates, using the **server clock**: authenticated → AAL2 → profile active → active membership → organization `pilot_active` → entitlement `active` → `now()` within the pilot window. The app calls it before real-data workflows and every ≤ 300 seconds while open.

AAL1 vs AAL2

- **AAL1** — password only. Sufficient to reach MFA enrollment/challenge pages; grants no real-data or console access.
- **AAL2** — password + a verified TOTP challenge in the current session. Required for real-data pilot access, all Operations Console access, and every privileged Edge Function.

Session revocation & propagation

Revocation is server-side (`auth.admin.signOut` , membership/entitlement status changes). An already-open browser session loses access at its next authorization revalidation — **maximum 300 seconds** — or immediately on any action that calls a privileged function. The app does not claim instant revocation.

Not built (future phases)

Full modularization (Phase 4), production Supabase project, production SMTP, backup/recovery automation, and monitoring are out of scope for R3 and are prerequisites for paid production.

5. Security Remediation (Authentication Design)

ARMP Security Remediation — R2 → R3

#	R2 weakness	R3 remediation	Verified
1	Client-side format-only pilot code (ARMP-PILOT-XXXX-XXXX) granted the pilot plan	Removed entirely; access requires a server authorization decision	Suite 10 + gate
2	Plan stored in localStorage; client could self-grant	launchApp blocks any non-demo plan without a server-authorized decision; revalidated ≤300 s	Suite 12
3	Operations Console gated by Google email-domain only	Supabase session + AAL2 + active internal role via list-operations-dashboard ; domain check removed	Suite 10
4	No MFA	Mandatory TOTP (AAL2) for pilot users, console users, and every privileged action	Local; staging E2E pending
5	No server authorization	RLS default-deny on all tables + authorize_pilot_session() server decision	RLS 26/26 local
6	No admin control over access lifecycle	20 Edge Functions: create/approve/activate/suspend/extend/complete/revoke pilots; invite/disable/enable/role/session/MFA-reset users	deno check; staging pending
7	No audit trail	Nonfinancial security_events + activation_events ; dual-control MFA reset	RLS + local

Residual risk (honest)

- Because the browser holds the working financial data, a user with local access can still see their own organization’s data — expected; RLS protects the control plane, not the user’s own screen.
- Revocation is not instantaneous; the bound is 300 seconds (documented, tested locally, staging-pending).
- All staging behaviors remain **NOT VALIDATED IN STAGING** until the E2E runs.

6. Pilot Access Workflow / Operations Console Guide

ARMP Pilot Access — Operations Console Guide

The console (`/admin.html`) requires a Supabase session, AAL2 (TOTP), and an active internal role. Owners and Operations Admins can perform write actions; Support and Sales/Read-Only are read-only (the server enforces this regardless of the UI).

Grant pilot access to a new organization

1. **Organizations → Create pilot organization** — enter legal/display name, approved ERP, seats. Creates the org in `pilot_pending` with a `pending` entitlement.
2. **Approve** — moves to `pilot_approved`.
3. **Activate** — enter pilot start/end dates. Moves to `pilot_active`; access opens automatically at the start date (server clock).
4. **Invite user** — enter business email, full name, customer role, expiration (1–30 days). The user gets a single-use, email/org/role-bound invitation.
5. The user sets a password, enrolls TOTP, and enters the pilot app once `authorize_pilot_session()` passes.

Lifecycle controls

- **Suspend** — immediate hold; users lose access within 300 s. **Restore** via Extend (with restore) or reactivation.
- **Extend** — forward-only new end date.
- **Complete** — ends the pilot (`pilot_completed`).
- **Revoke** — terminal; entitlement revoked, org terminated, all member sessions signed out server-side.
- **Users** — disable/enable, change role, revoke sessions, or start a **dual-control MFA reset** (a different admin must complete it).

What the console never shows

No invoice, bank, remittance, matching, or allocation data — that never leaves the browser and is never stored in Supabase.

7. Supabase Owner Setup

ARMP Supabase Owner Setup — Staging

These are the **owner-only** steps that cannot be done from code or CI. Do them in order. **Never paste any secret value into chat.** All values below are either public or entered only in the Supabase/GitHub dashboards.

| Step 1 — Auth URL configuration ✓ (confirmed done)

Authentication → URL Configuration. Site URL = `https://armatchpro-staging.netlify.app`; the nine staging redirect URLs added. Production URLs prepared but NOT added until production approval.

| Step 2 — GitHub repo secrets (names only; values entered in GitHub, never shared)

GitHub → repo `wballard82/ar-match-pro` → Settings → Secrets and variables → Actions → New repository secret. Add: - `SUPABASE_ACCESS_TOKEN` — from Supabase → Account → Access Tokens → Generate. Secret. Do not share. - `SUPABASE_DB_PASSWORD` — your staging database password (Supabase → Project Settings → Database). Secret. Do not share. - `NETLIFY_AUTH_TOKEN` — optional, only if CI deploys the static site. Secret.

Confirm success: the three names appear in the Actions secrets list. Values are write-only in GitHub — that’s expected.

| Step 3 — Push the R3 branch

Claude’s environment cannot push (no Git credentials). You will receive `armp-r3.bundle`. From a machine with repo access:

```
git clone https://github.com/wballard82/ar-match-pro && cd ar-match-pro
git bundle verify /path/to/armp-r3.bundle
git fetch /path/to/armp-r3.bundle pilot-auth-r3:pilot-auth-r3
git push origin pilot-auth-r3
```

Confirm success: branch `pilot-auth-r3` shows on GitHub. `main` is unchanged.

| Step 4 — Deploy migrations + functions to staging (after you approve the file list)

GitHub → Actions → “ARMP R3 — Staging Deploy (manual)” → Run workflow → branch `pilot-auth-r3`, type `STAGING` to confirm. This runs `supabase db push` and `supabase functions deploy` against ref `vjxdqmujsxmllfvnksvpy`. **Confirm success:** workflow is green; Supabase → Database → Tables shows the ARMP tables; Edge Functions shows 20 functions.

| Step 5 — First owner bootstrap (one-time)

- Supabase → Edge Functions → Secrets (or `supabase secrets set`): add `ARMP_BOOTSTRAP_ENABLED = true`. Secret-ish; do not share.
- Supabase → Authentication → Users → Add user → your ARMP email + a strong password (enter it only in the dashboard).
- Sign in at `/admin.html`, complete TOTP enrollment, then the app calls `bootstrap-first-owner`, assigning you `armp_owner`.
- Immediately** remove `ARMP_BOOTSTRAP_ENABLED` (set it to empty / delete it). The function is then permanently inert. **Confirm success:** you can load the Operations Console; Activity shows `owner_bootstrap_completed`.

| Step 6 — SMTP (later, for reliable invitations)

Supabase’s default email is rate-limited (~2-4/hour) and fine for early staging. For real invitation volume: Authentication → Email → SMTP Settings → enter host, port, user, and password (password entered only in the dashboard; never shared). **Confirm success:** a test invitation email arrives within a minute.

8. Support Procedures

ARMP Support Procedures — R3

- **Support role** (`armp_support`) can view the Operations Console (read) but cannot activate/suspend/extend/complete/revoke pilots or change entitlements — the server enforces this.
- **Password reset:** direct the user to the sign-in page's reset link (`/auth/reset-password.html`); they re-verify TOTP after reset.
- **Lost authenticator:** an Operations Admin starts a **dual-control MFA reset** (`request-mfa-reset`); a *different* admin completes it (`complete-mfa-reset`), which removes TOTP factors and signs the user out to re-enroll.
- **“I can't get in” triage:** confirm invitation accepted → profile active → org `pilot_active` → within pilot dates → membership active. The console's Users/Activity views show each.
- **Never** ask a user for their password, TOTP secret, or recovery codes.

9. Authentication Test Report

ARMP Authentication Test Report — R3

Scope

Password auth, TOTP MFA (enrollment + challenge), AAL1/AAL2 enforcement, invitation acceptance, session revocation.

Local / static results RUN

- Frontend security (Suite 10, 176/176): auth pages present; `app.html` requires a server decision; console gate uses Supabase session + AAL2 + internal role; no secrets/format-codes in frontend.
- R2-migration (Suite 12, 18/18): legacy localStorage cannot authorize; failed auth never falls back to Demo Mode; \$13 messages surface; workspace data preserved.
- Authorization function AAL gate: AAL1 denied, AAL2 authorized (proven in the RLS suite against real Postgres).

Staging results NOT RUN

The following require the live project and are **NOT VALIDATED IN STAGING**: real password sign-in, TOTP enrollment against Supabase Auth, TOTP challenge, invitation email delivery + acceptance, `auth.admin.signOut` propagation timing. These will be executed and reported per-step in the Staging Acceptance Report.

10. RLS Test Report

ARMP RLS Test Report — R3

Method

`supabase/tests/local_shim.sql` replicates the Supabase runtime surface (`auth.uid()`, `auth.jwt()`, `anon/authenticated/service_role` roles) on real PostgreSQL 16. All 12 migrations apply, then `rls_test.sql` runs as each role via `SET ROLE` + JWT-claim GUCs — exactly as PostgREST does.

Result 26 / 26 PASS on real Postgres 16

Proven: anonymous denied on all 7 protected tables; customer A sees only its own org + entitlement and cannot see or write Corp B; customer cannot change pilot dates/status, insert an internal role, or change own role; `internal_notes` column not selectable by customers; AAL1 denied and AAL2 authorized via `authorize_pilot_session()`; ops admin reads across orgs, invitations, and roles.

Label

These are **local-Postgres** results. Re-running RLS against the live staging project is part of the acceptance battery (the CI `staging-tests` workflow reproduces this suite on an ephemeral Postgres and can be pointed at staging).

11. Data Boundary Test Report

ARMP Data-Boundary Test Report — R3

Method

The packaged `deploy/app.html` boots in jsdom with `fetch`, `XMLHttpRequest`, `sendBeacon`, and `WebSocket` instrumented. A real customer-derived invoice fixture (2,226 rows) is imported through the template adapter, a bank payment is created, matching runs, and export is invoked. Every network attempt is captured and scanned.

Result 8 / 8 PASS (local)

- No network call carries invoice numbers, POs, SOs, accounts, customer names, or amounts.
- Only Supabase auth/functions destinations are permissible; no other origins attempted.
- No spreadsheet/base64 file payload in any request body; no WebSocket connections; export stayed local.

Label

Local jsdom proof of the frontend’s behavior. The staging data-boundary run (same instrumentation against the live site) is part of the acceptance battery and is **NOT YET RUN**.

12. Test Results

ARMP Test Results — ARMP-2026.07.21-PILOT-AUTH-R3

JavaScript battery (run locally this build) RUN

Suite	Assertions
1 — \$232,050 real-data E2E	26/26
2 — Review controls	24/24
3 — Import Template & validation	23/23
4 — Multi-file regression & smoke	15/15
5 — TK-Fujikin \$2,305,295 template E2E	34/34
6 — OCR-degraded RMA recommendation	9/9
7 — Go-live guardrails	30/30
8 — Support bot	24/24
9 — Prohibited-pattern gate (R3, expanded)	1063/1063
Video player	27/27
Staging smoke (deployable artifact)	8/8
10 — Frontend security (R3)	176/176
11 — Data boundary (network-instrumented)	8/8
12 — R2 localStorage cannot grant access	18/18

R2 regression subset (suites 1-9 + video + staging_smoke): 503/503 — unweakened.

Database / RLS (real PostgreSQL 16) RUN

- 12 migrations apply cleanly; `rls_test.sql` **26/26 PASS**.

Edge Functions TYPE-CHECK / RUNTIME NOT RUN

- 20 functions type-check under Deno with real dependency resolution.
- Runtime behavior against live Supabase Auth: **NOT VALIDATED IN STAGING**.

Staging acceptance NOT RUN

See ARMP_STAGING_ACCEPTANCE_REPORT — every step NOT RUN until executed against the live project.

13. Prohibited-Pattern Sweep

ARMP Prohibited-Pattern Sweep — R3

The sweep is executable: `tests/test_prohibited_terms.js` (Suite 9). Result this build: **1063/1063 PASS**, and a planted-violation negative control correctly failed the gate (1062/1063), proving it is not a no-op.

Zero-tolerance in deployables + generators

MASTER_ADMIN(_EMAIL/PASS), VALID_KEYS, ARMP-PILOT-XXXX-XXXX, isValidPilotCode, rows: Infinity, Free (7-day), 7-day demo, free trial, no IT approval, all Stripe domains, stripeCustomerId/SubscriptionId, checkout=success, pk_live, Stripe price IDs, query-string plan entitlement, localStorage plan authorization.

Secret-VALUE patterns (deployables AND backend)

sb_secret_*, embedded JWT literals, SUPABASE_SERVICE_ROLE_KEY=<value>, database_password=<value>, JWT_SECRET=<value>, SMTP/DB passwords with values.

\$24 context-awareness (not a naive global scan)

- `service_role`, `using (true)`, `with check (true)` are **permitted in backend source** and are NOT in the deployable ban list.
- The gate proves `service_role` appears in backend and in **no** frontend file.
- It distinguishes a variable **name** from a secret **value**, and a test that **searches** for a pattern from code that **implements** it.
- No broad whitelists; the only carve-outs are the backend keyword allowances above.

14. Deployment

ARMP Deployment — R3

Frontend (Netlify staging)

Upload the contents of `deploy/` (including `deploy/auth/`) to the Netlify staging site root. `_headers` must be at the root. **Never upload** `tests/`, `supabase/seed/`, `source/internal/`, `generators/`, or `docs/` — they contain customer-derived fixtures or internal material.

Frontend config uses only public values: `SUPABASE_URL=https://vjxdmujxnmlfvnksvpy.supabase.co` and the publishable key. The service-role key is never in frontend config.

Backend (Supabase staging) — owner-approved, CI-driven

1. Owner sets GitHub secrets (`ARMP_SUPABASE_OWNER_SETUP`, Step 2).
2. GitHub → Actions → “ARMP R3 — Staging Deploy (manual)” → type `STAGING`. Runs `supabase db push` + `supabase functions deploy` against ref `vjxdmujxnmlfvnksvpy`.
3. Bootstrap the first owner (Step 5), then unset `ARMP_BOOTSTRAP_ENABLED`.

Exact files deployed to staging (shown before any apply, per §27A)

- Migrations: `supabase/migrations/20260722000000{1..8}*.sql`, `..8a`, `..9`, `..10`, `..11` (12 files).
- Functions: all 20 directories under `supabase/functions/` + `_shared/armpt.ts`. Command target: project ref `vjxdmujxnmlfvnksvpy` (staging).
Production is never targeted.

CSP / headers

`deploy/_headers` allows the Supabase origin in `connect-src` (https only), no Stripe origins, `payment=()`, `object-src none`, `base-uri/form-action restricted`, `referrer-policy set`, `nosniff`. The `_headers` HTTP response file — not a meta tag — is the primary control.

Verify

`sha256sum -c SHA256SUMS.txt`, then `npm install && npm test`. Smoke-check `/`, `/demo.html` (synthetic), `/admin.html` (console sign-in), `/auth/*` pages.

15. Rollback

ARMP Rollback — R3

Never roll back to

- Any release with hardcoded credentials (pre-R2).
- Any release with exposed Stripe identifiers (pre-R2).
- Format-only pilot access (R2) for **real customer data**.

Sanitized rollback baseline

`ARMP_SANITIZED_ROLLBACK_BASELINE.zip` = the R2 **deploy set** (no credentials, no Stripe, no format-code real-data access is enabled without the R3 backend). Use it only for the **public website / Demo Mode**, never to grant real-data pilots.

Frontend rollback (Netlify)

Netlify → Deploys → select last-known-good → Publish deploy. Atomic; the site is static.

Backend rollback (Supabase)

- Migrations are forward-only in practice: prefer a **forward fix** migration over destructive down-migrations. RLS changes must be corrected with a tested forward migration, never by disabling RLS.
- Auth and invitation state must not be corrupted: do not delete `auth.users` rows to “reset”; use the disable/revoke Edge Functions.
- Control-plane metadata (orgs, entitlements, events) should be backed up (Supabase scheduled backups) before any schema change.
- Financial workspace data is local to each browser and is unaffected by any rollback.

Different actions for different planes

Public-website rollback (Netlify redeploy) and authenticated pilot-app rollback (Netlify + Supabase migration/function state) are separate operations; performing one does not roll back the other.

16. Incident Response

ARMP Incident Response — R3 (staging)

Immediate containment

- **Compromised pilot user:** Operations → Users → Disable + Revoke sessions (server-side sign-out). Propagation ≤ 300 s.
- **Compromised organization:** Revoke pilot access (terminal): entitlement revoked, org terminated, all member sessions signed out.
- **Suspected key exposure:** rotate the service-role key in Supabase (Project Settings → API), redeploy functions. The publishable key is public and needs no rotation. Never share keys in chat/tickets.

Investigation

Use `security_events` + `activation_events` (nonfinancial) via the Operations Console Activity view; every privileged action carries a `request_id`.

Boundaries

No customer financial data is stored server-side, so a control-plane incident does not expose invoice/payment data. Financial workspace data lives in each user's browser.

Escalation & notification

Document scope, affected orgs/users, and timeline. Legal/notification obligations are determined with counsel (legal drafts pending attorney review).

17. Claims Register

ARMP Claims Register — R3

Claim	Status	Basis
“Server-controlled pilot access”	Supportable after staging	RLS + Edge Functions + authorize_pilot_session; local-verified, staging pending
“Mandatory MFA for pilot and console users”	Supportable after staging	AAL2 enforced in gate + functions; staging E2E pending
“Financial data stays in your browser”	Supportable	Data-boundary suite 8/8 local; staging repeat pending
“Organization isolation”	Supportable after staging	RLS 26/26 local; staging RLS pending
“Immediate revocation”	NOT claimed	Revocation is ≤ 300 s, not instant — stated honestly
“Paid-production ready”	NOT claimed	Explicitly blocked
“No ERP integration or implementation project required”	Supportable	Approved wording; unchanged from R2

18. Consistency Matrix

ARMP Consistency Matrix — R3

Claim	app	admin	auth pages	migrations	functions	docs
Server authorization required	✓	✓	✓	✓	✓	✓
Mandatory TOTP (AAL2)	✓	✓	✓	✓ (session_aal)	✓ (privileged)	✓
No client-side entitlement	✓	✓	n/a	n/a	n/a	✓
No Stripe anywhere	✓	✓	✓	✓	✓	✓
Financial data local only	✓	✓ (never stores)	n/a	✓ (no fin tables)	✓ (no fin data)	✓
Revocation ≤300 s (not instant)	✓	✓	n/a	n/a	✓	✓
service_role backend-only	✓ (absent)	✓ (absent)	✓ (absent)	n/a	✓ (env only)	✓

No contradictions found across the deployable set and documentation.

19. File Audit

ARMP File Audit — R3 (authoritative file per purpose)

Purpose	Authoritative file	Status
Pilot app	deploy/app.html	Server-gated; pilot-only; not paid-production
Operations Console	deploy/admin.html	Supabase + AAL2 + internal role
Auth pages	deploy/auth/{invite,callback,setup-mfa,challenge-mfa,reset-password}.html	MFA-enforcing
Public site	deploy/{index,demo,video,terms,privacy,checkout}.html	Public / synthetic Demo
Headers	deploy/_headers	Supabase origin; no Stripe
DB schema	supabase/migrations/*.sql (12)	Real-Postgres-verified
RLS	supabase/migrations/20260722000009_rls_policies.sql	Default-deny; 26/26
Edge Functions	supabase/functions/* (20 + _shared)	deno-checked
Authorization	supabase/migrations/20260722000011_authorize_fn.sql	Server decision
Import template	templates/ARMP_Invoice_Import_Template.xlsx	7 required fields
Tests	tests/* (14 suites)	503 R2 + R3 additions
Release gate	tests/test_prohibited_terms.js	Context-aware (R3)

Superseded R2 items (format-code, license inputs) are removed, not carried forward.

20. Known Limitations & Remaining Blockers

ARMP Known Limitations — R3

- **Staging unvalidated in this package.** Every staging behavior (migrations applied, functions live, MFA, invitations, revocation timing, E2E) is NOT VALIDATED IN STAGING until the acceptance battery runs against project `vjxdqmujsxnmLfvnksvpy`.
- **Revocation latency.** Server-side revocation propagates to an open session at its next revalidation — up to 300 seconds — not instantly.
- **Browser-local data is visible to the signed-in user.** RLS protects the control plane; it does not (and cannot) hide a user's own organization's financial data on their own screen.
- **Edge Functions are type-checked, not yet runtime-tested** against live Supabase Auth (invites, MFA factor deletion, global sign-out).
- **Supabase default email is rate-limited** (~2-4/hour); production SMTP is required for real invitation volume.
- **Legal drafts require attorney review;** pricing is not final.
- **No production Supabase project, backup/recovery automation, or monitoring** — all prerequisites for paid production.

21. Staging Acceptance Report

ARMP Staging Acceptance Report — R3

Status: **NOT RUN**. No step below has been executed against the live Supabase staging project or Netlify staging site. This report is the template that the CI workflows and manual E2E will populate. No step may be marked passed until it actually runs.

Automated (CI staging-tests workflow)

Step	Status
Migrations apply to staging	NOT RUN
RLS suite against staging-equivalent Postgres	NOT RUN (26/26 local)
503 R2 regression	NOT RUN in CI (503/503 local)
Frontend-security / data-boundary / R2-migration suites	NOT RUN in CI (pass local)
Prohibited-pattern gate	NOT RUN in CI (pass local)

16-step real-data E2E (manual + MFA)

#	Step	Status
1	Owner signs into Operations Console (AAL2)	NOT RUN
2	Create pilot organization	NOT RUN
3	Approve pilot	NOT RUN
4	Set start/end dates (activate)	NOT RUN
5	Invite user	NOT RUN
6	User opens invitation	NOT RUN
7	User creates password	NOT RUN
8	User enrolls TOTP	NOT RUN
9	User completes MFA	NOT RUN
10	User enters real-data app	NOT RUN
11	Admin suspends pilot	NOT RUN
12	User loses access (≤300 s)	NOT RUN
13	Admin restores/extends	NOT RUN
14	User regains access after reauth	NOT RUN
15	Admin revokes user	NOT RUN
16	User cannot regain access via localStorage or prior session	NOT RUN

Decision

Controlled real-data pilot access: **NOT APPROVED** until every step above is PASS and an independent review is complete.